

GETTING INFORMATION			
ACTION	IP / OTHERS	NMCLI	READING FILE
INTERFACES	ip a	nmcli device show	—
GATEWAY	ip route grep default	nmcli device show grep GATEWAY	—
ROUTES	ip route	nmcli device show grep IP4.route	—
DNS	resolvectl status	nmcli device show grep IP4.DNS	cat /etc/resolv.conf
HOSTNAME	hostnamectl	nmcli general hostname	cat /etc/hostname
ip is instantaneous (ip addr add, ip route add); nmcli (nmcli con mod) and netplan apply stays after reboot; ip don't. ADVANCED: To see actual routes table, use: cat /proc/net/route			

BASIC DIAGNOSTICS	
Lost in the troubleshooting? Start from the ground up — check each layer before moving to the next.	
ACTION	COMMAND
INTERFACE UP AND HAS IP?	ip a
DO I REACH ROUTER?	ping -c 4 <gateway ip>
DO I REACH INTERNET?	ping 8.8.8.8
IS DNS SOLVING?	dig example.com (nslookup example.com)
EVERYTHING IS OK?	ping google.com

SETTING PROPERTIES		
ACTION	IP / OTHERS	NMCLI
ACTIVATE/RESTART INTERFACE	ip link set eth0 up down	nmcli con up down eth0
STATIC IP/MASK	ip addr add 192.168.1.10/24 dev eth0	nmcli con mod eth0 ipv4.addresses 192.168.1.10/24
REMOVE IP	ip addr del 192.168.1.10/24 dev eth0	nmcli con mod eth0 ipv4.addresses ""
SETTING DHCP	dhclient eth0 (needs ISC dhclient)	nmcli con mod eth0 ipv4.method auto manual
GATEWAY	ip route add default via 192.168.1.1	nmcli con mod eth0 ipv4.gateway 192.168.1.1
ADD STATIC ROUTES	ip route add 10.0.0.0/24 via 192.168.1.254	nmcli con mod eth0 +ipv4.routes "10.0.0.0/24 192.168.1.254"
REMOVE ROUTES	ip route del 10.0.0.0/24	nmcli con mod eth0 -ipv4.routes "10.0.0.0/24 192.168.1.254"
DNS	resolvectl dns eth0 8.8.8.8	nmcli con mod eth0 ipv4.dns "8.8.8.8 1.1.1.1"
DNS DOMAIN SEARCH	resolvectl domain eth0 example.com	nmcli con mod eth0 ipv4.dns-search example.com
HOSTNAME	hostnamectl set-hostname new-name	nmcli general hostname new-name
APPLYING CHANGES	Immediately, but it's overwritten after reboot	nmcli con up eth0
ip: change the kernel state on time but doesn't persist after reboot. nmcli: Write a connection profile, saving configuration after reboot (by NetworkManager).		



HOW TO FIND WHAT IS THE ACTIVE NETWORK MANAGER

To discover what is the network manager, use:

```
sudo systemctl is-active NetworkManager
sudo systemctl is-active systemd-networkd
sudo systemctl is-active systemd-resolved
```



NOTES

Most commands here require root privileges — sudo was omitted in several places for readability. Placeholders like <interface>, <ip>, <gateway> must be replaced with real values.

This cheatsheet assumes a standard Debian setup with NetworkManager. Commands may vary slightly depending on your distro version or network manager in use.

STATIC IP - NMCLI

BASH

```
1 # Use this command to discover the connection name (CONNECTION column).
2 # If the device STATE is unmanaged, read below.
3 sudo nmcli device
4 # If no connection exists yet, create one
5 sudo nmcli c add type ethernet con-name <name> ifname <interface>
6
7 sudo nmcli c m <name> ipv4.addresses <ip>/<mask>
8 sudo nmcli c m <name> ipv4.gateway <gateway>
9 sudo nmcli c m <name> ipv4.method manual
10 sudo nmcli c m <name> ipv4.dns "<dns1> <dns2>"
11 sudo nmcli c m <name> ipv4.dns-search "domain.test"
12 sudo nmcli c down <name>
13 sudo nmcli c up <name>
14
15 # Confirm
16 nmcli device show <interface>
```

If the STATE of interface is unmanaged, try using netplan.

STATIC IP - NETPLAN

Netplan is the network declarative configuration layer, for NetworkManager or systemd-networkd. Create a file in /etc/netplan/test.yml:

YAML

```
1 network:
2   version: 2
3   renderer: networkd # or 'NetworkManager', depends on backend
4   ethernets:
5     eth0:
6       dhcp4: false # if true, ignores addresses/routes/gateway
7       addresses:
8         - 192.168.1.10/24 # IP/mask (CIDR)
9       routes:
10        - to: default
11          via: 192.168.1.1 # default gateway
12        - to: 10.0.0.0/24
13          via: 192.168.1.254 # additional static route
14       nameservers:
15         addresses: [8.8.8.8, 1.1.1.1] # DNS servers
16         search: [exemplo.com] # DNS search domain
```

Give permissions to all configuration files:
sudo chown root:root /etc/netplan/*.yaml
sudo chmod 600 /etc/netplan/*.yaml
Then to activate/restart interface, use netplan apply

DNS

DNS translates domain names into IP addresses.

ACTION	COMMAND
ADD INDIVIDUAL HOST	echo "192.168.1.10 client.domain.test client" sudo tee -a /etc/hosts
SET DNS SERVER	sudo nmcli c m <name> ipv4.dns 192.168.0.100
IGNORE AUTO DNS (DHCP)	sudo nmcli c m <name> ipv4.ignore-auto-dns yes
SET DNS SEARCH DOMAIN	sudo nmcli c m <name> ipv4.dns-search "dominio.test"
APPLY CHANGES	sudo nmcli c down <name> && sudo nmcli c up <name>
FLUSH DNS CACHE (systemd-resolved)	sudo resolvectl flush-caches
REAPPLY DNS (NetworkManager)	sudo systemctl restart NetworkManager

To set DNS directly, edit /etc/resolv.conf
To lock /etc/resolv.conf against NetworkManager overwrite: sudo chattr +i /etc/resolv.conf

ACTION	COMMAND	SIMILAR COMMAND
What DNS server am I using?	resolvectl status	cat /etc/resolv.conf
Can I resolve this domain?	dig example.com	nslookup example.com
Does this specific DNS server work?	dig example.com @8.8.8.8	nslookup example.com 8.8.8.8
What domain does this IP belong to?	dig -x 8.8.8.8	nslookup 8.8.8.8
Is DNS slow?	dig example.com grep "Query time"	—
Just give me the IP	dig +short example.com	—
Where is resolution failing?	dig +trace example.com	—
What are the MX/NS/TXT records?	dig example.com MX	dig example.com NS
What does the system actually resolve?	getent hosts example.com	—

SETTING WI-FI WITH NMCLI

```
BASH
1 sudo nmcli radio wifi on # enable wifi radio
2 nmcli device set <interface> managed yes # let NetworkManager manage
3 nmcli device wifi list # scan/list networks
4 nmcli device wifi connect "<SSID>" password "<PASS>" # connect
5 sudo systemctl restart NetworkManager # restart service
```

To add another network to nmcli, use: `nmcli device wifi connect "<SSID2>" password "<PASS2>"`
This creates a new profile automatically. List all profiles with: `nmcli connection show`

SETTING WI-FI WITH IP / WPA SUPPLICANT

```
BASH
1 sudo ip link set wlan0 up # enable interface
2 sudo iw dev wlan0 scan | grep SSID # scan networks
3 wpa_passphrase "<SSID>" "<PASS>" | sudo tee /etc/wpa_supplicant.conf # generate config
4 sudo wpa_supplicant -B -i wlan0 -c /etc/wpa_supplicant.conf # connect
5 sudo dhclient wlan0 # get IP via DHCP
```

WI-FI VIA NETPLAN

```
BASH
1 network:
2   version: 2
3   renderer: NetworkManager # wifi generally uses NetworkManager as backend
4   wifis:
5     wlan0:
6       dhcp4: true # get IP via DHCP
7       access-points:
8         "SSID1": # add how many you wish
9           password: "password1"
```

Apply with: `netplan apply`. If IP needs to be static, see how to do in Netplan static configuration example.

WI-FI ADVANCED STATUS

NETWORKMANAGER LOGS	<code>journalctl -u NetworkManager -f</code>
AUTHENTICATION LOGS	<code>journalctl -u wpa_supplicant -f</code>
PCI CHIPSET/DRIVER	<code>lspci -k grep -A3 Network</code>
USB CHIPSET	<code>lsusb -v</code>
DRIVER BOOT ERRORS	<code>dmesg grep -i wlan</code>

WI-FI TROUBLESHOOTING

ACTION	COMMAND	SIMILAR COMMAND
Is the interface up?	<code>ip link show wlan0</code>	<code>nmcli device status</code>
Are there networks nearby?	<code>nmcli device wifi list</code>	<code>iw dev wlan0 scan grep SSID</code>
What am I connected to?	<code>nmcli device show wlan0</code>	<code>iw dev wlan0 link</code>
Is the signal weak?	<code>iwconfig wlan0</code>	<code>iw dev wlan0 station dump grep signal</code>
Am I getting an IP?	<code>ip a show wlan0</code>	<code>nmcli device show wlan0 grep IP4.ADDRESS</code>
Can I reach the gateway?	<code>ping -c 4 \$(ip route grep default awk '{print \$3}')</code>	—
Can I reach the internet?	<code>ping -c 4 8.8.8.8</code>	<code>mtr 8.8.8.8</code>
Is it a DNS problem?	<code>dig exemplo.com</code>	<code>resolvectl status</code>
What does the driver/hardware say?	<code>dmesg grep -i wlan</code>	<code>lspci -k grep -A3 Network</code>
What is NetworkManager doing?	<code>journalctl -u NetworkManager -f</code>	<code>journalctl -u wpa_supplicant -f</code>

To install iwconfig and iw, use: `sudo apt install iw wireless-tools`

VLAN/BRIDGE/BONDING (VIRTUAL INTERFACES)

VLAN segments a physical network into isolated virtual networks using tagged interfaces.
Bridge connects multiple interfaces into a single logical network (common in VMs/containers).
Bonding aggregates multiple physical interfaces for redundancy or increased throughput.

ACTION	COMMAND
Create VLAN interface	sudo ip link add link eth0 name eth0.10 type vlan id 10
Create bridge	sudo ip link add br0 type bridge
Add interface to bridge	sudo ip link set eth0 master br0
Create bonding (link aggregation)	sudo modprobe bonding && sudo ip link add bond0 type bond
List VLAN/bridge interfaces	ip -d link show

SS & NETSTAT

See what's connected, listening, or waiting — ss shows active sockets and connections, useful for spotting open ports, established sessions, or unexpected traffic.

ACTION	SS	NETSTAT
List all active connections	ss -tun	netstat -tun
Show connections with process	sudo ss -tulnp	sudo netstat -tulnp
Count connections by state	ss -antH awk '{print \$1}' sort uniq -c	—
Show only established connections	ss -t state established	netstat -t grep ESTABLISHED
Show only listening ports	ss -tuln	netstat -tuln
Filter by port	ss -tuln grep :443	netstat -tuln grep :443
Show connections to a specific IP	ss dst 192.168.1.10	netstat -tn grep 192.168.1.10

To use netstat, install: `sudo apt install net-tools`

ARP

ARP maps IP addresses to MAC addresses on the local network. The kernel keeps this mapping cached — checking or editing it helps troubleshoot local connectivity and detect spoofing.

ACTION	IP NEIGH	ARP
Local ARP table (known IP↔MAC)	ip neigh	arp -a
Find MAC of a specific IP	ip neigh show to <ip>	arp -n <ip>
Add static ARP entry	ip neigh add <ip> lladdr <macaddress> dev <interface>	—
Delete/flush ARP entry	ip neigh flush to <ip>	—

To use arp, install: `sudo apt install net-tools`

PROXY

A proxy sits between your machine and the internet, forwarding requests on your behalf — often used to bypass restrictions, cache traffic, or route through a specific network.

Set proxy for current shell session	export http_proxy="http://proxy:3128"	export https_proxy="http://proxy:3128"
Set for apt	echo 'Acquire::http::Proxy "http://proxy:3128";' sudo tee /etc/apt/apt.conf.d/95proxy	—
Set system-wide proxy (persistent)	edit /etc/environment → add http_proxy=...	—
Check proxy env vars	env grep -i proxy	printenv grep -i proxy
Test if proxy is being used	curl -v https://example.com 2>&1 grep -i proxy	—
Bypass proxy for specific hosts	export no_proxy="localhost,127.0.0.1,.local"	—

NETWORK TROUBLESHOOTING

IS THERE ANY CONNECTIVITY AT ALL?	ping -c 4 8.8.8.8	PORT SEEMS CLOSED BUT SERVICE IS RUNNING?	sudo ss -tulnp grep <PORT>
SEE DEFAULT GATEWAY	ip route grep default	TRAFFIC ISN'T REACHING A SERVICE?	sudo tcpdump -i eth0 port <PORT>
CHECK IF GATEWAY RESPONDS	ping -c 4 \$(ip route grep default awk '{print \$3}')	FIREWALL BLOCKING OUTBOUND TRAFFIC?	sudo iptables -L OUTPUT -n -v
TEST WEBSITE ACCESS	curl -I https://example.com	NAT/MASQUERADE NOT WORKING?	sudo iptables -t nat -L -n -v
TEST THROUGH PROXY	curl -x http://proxy:3128 -I https://example.com	VM/CONTAINER HAS NO NETWORK?	ip -d link show
TRACE ROUTE TO HOST (HOP BY HOP)	traceroute 8.8.8.8	HIGH LATENCY OR PACKET LOSS?	sudo iftop -i eth0
TRACE + LATENCY PER HOP	mtr -rw 8.8.8.8	BANDWIDTH HOG ON THE NETWORK?	sudo iftop -i eth0

FIREWALL/NAT

FIREWALL: Filters network traffic based on rules (allow/deny/reject) by port, protocol, IP, or interface, protecting the system from unwanted connections.		
NAT: Translates IP addresses/ports between networks — commonly used to share one public IP among multiple internal hosts (masquerade) or route traffic to internal services.		
IPTABLES: Low-level Linux firewall tool managing netfilter tables directly (filter, nat, mangle). More flexible but verbose syntax; rules aren't persistent by default.		
UFW: Simplified frontend for iptables. Enables/disables the firewall, sets default policies, adds rules by port/protocol/IP/interface, supports rate limiting and app profiles — rules persist automatically.		
ACTION	IPTABLES	UFW
List rules	sudo iptables -L -n -v / sudo nft list ruleset	sudo ufw status verbose
Allow a port	sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT	sudo ufw allow 22/tcp
Block an IP	sudo iptables -A INPUT -s 1.2.3.4 -j DROP	sudo ufw deny from 1.2.3.4
Allow traffic on specific interface (ports 80/443)	sudo iptables -A INPUT -i eth0 -p tcp -m multiport --dports 80,443 -j ACCEPT	sudo ufw allow in on eth0 to any port 80,443 proto tcp
NAT/masquerade (share internet)	sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE	edit /etc/ufw/before.rules manually
Save rules persistently	sudo netfilter-persistent save	automatic (ufw enable persists)
Enable firewall	—	sudo ufw enable
Remove a rule	sudo iptables -D INPUT -p tcp --dport 22 -j ACCEPT	sudo ufw delete allow 22/tcp

NETWORK HARDENING

Default deny incoming (for whitelist)	sudo ufw default deny incoming
Allow only needed ports (for whitelist)	sudo ufw allow 22,80,443/tcp
Restrict access by interface	sudo ufw allow in on eth0 to any port 80,443 proto tcp
Restrict access by source IP	sudo ufw allow from 192.168.1.0/24
Rate-limit SSH connections	sudo ufw limit ssh
Disable unused network protocols	blacklist in /etc/modprobe.d/ (e.g. dccp, sctp)
Lock /etc/resolv.conf against overwrite	sudo chattr +i /etc/resolv.conf
Use trusted DNS servers only	set nameservers via nmcli/netplan (avoid ISP default)
Disable IP forwarding (if not router)	net.ipv4.ip_forward=0
Enable SYN flood protection	net.ipv4.tcp_syncookies=1
Disable ICMP redirects	net.ipv4.conf.all.accept_redirects=0
Ignore broadcast pings	net.ipv4.icmp_echo_ignore_broadcasts=1

OTHER HARDENING (NOT COVERED HERE)

See other types of hardening not covered on this cheatsheet:	
Disable unused services	Reduces attack surface — a stopped service can't be exploited.
Restrict sudo access	Fewer root-capable users means fewer ways to compromise the whole system.
Enable AppArmor/SELinux	Confines processes, limiting damage even if one is compromised.
Use Fail2ban	Bans IPs automatically after repeated failed login attempts.
Use 2FA (Google Authenticator PAM)	Second layer of verification — a leaked password/key alone isn't enough.
Set account lockout policies (pam_tally2/faillock)	Blocks accounts after repeated failed logins, slowing brute-force.
Monitor ports (ss/nmap)	Detects unexpected services or backdoors.
Centralize logs (rsyslog, Wazuh)	Keeps logs safe even if the server is compromised.
Use VPN (WireGuard/OpenVPN) for admin access	Keeps SSH/panels off the public internet.
Segment network (VLANs/firewalls)	Limits lateral movement from a compromised host.

NETWORK SCANNING

Network scanning helps you discover what's alive on a network, which ports are open, and what services are running — useful for inventory, troubleshooting, and security audits. nmap is the standard tool for this, but there are faster/simpler alternatives depending on the goal: arp-scan and netdiscover work at Layer 2 (fast, local network only), while nmap works at Layer 3+ (routable, more detailed, but slower). Always scan networks you own or have permission to scan.

Is a specific host alive?	ping -c 4 192.168.1.10
Discover hosts on local network (Layer 2, fast)	sudo arp-scan --localnet
Discover hosts on a subnet (no ports)	sudo nmap -sn 172.16.38.0/23
List only IPs that are up, sorted	sudo nmap -sn -n 172.16.38.0/23 -oG - awk '/Status: Up/ {print \$2}' sort -V
Count total hosts found	sudo nmap -sn -n 172.16.38.0/23 -oG - awk '/Status: Up/ {print \$2}' sort -V awk 'BEGIN{c=0} {print; c++} END{print "Total hosts:", c}'
Scan common ports on a single host	nmap 192.168.1.10
Scan specific ports across a subnet	sudo nmap -n -p 22,80 172.16.38.0/23
List hosts with specific ports open, sorted + counted	sudo nmap -n -p 22,80 172.16.38.0/23 -oG - awk '/Status: Up/ {print \$2}' sort -V awk 'BEGIN{c=0} {print; c++} END{print "Total hosts:", c}'
Identify service/version on open ports	nmap -sV 192.168.1.10
Detect OS of target	sudo nmap -O 192.168.1.10
Full aggressive scan (OS+version+scripts+traceroute)	sudo nmap -A 192.168.1.10
Fast scan of a large range	sudo masscan -p22,80,443 172.16.38.0/23 --rate=1000

To use arp-scan and netdiscover, use: `sudo apt install arp-scan netdiscover masscan`.

TCPDUMP (PACKET CAPTURE)

Tcpdump captures and inspects network packets in real time, using filters by host, port, or protocol.

ACTION	COMMAND
Capture on an interface	sudo tcpdump -i eth0
Capture with more detail	sudo tcpdump -i eth0 -v
Filter by host	sudo tcpdump -i eth0 host 192.168.1.10
Filter by port	sudo tcpdump -i eth0 port 443
Save capture to file	sudo tcpdump -i eth0 -w capture.pcap
Read saved capture	tcpdump -r capture.pcap

To use tcpdump (if not installed), use:
`sudo apt install tcpdump`

AGRESSIVE SCANS | ARP SPOOFING | MITM

Script scan (vulns)	nmap --script vuln <ip>
Banner grabbing (identify service)	nmap -sV --version-intensity 5 <ip>
Stealth scan (SYN, evade logging)	nmap -sS <ip>
Evade firewall (fragment packets)	nmap -f <ip>
Spoof source IP	nmap -S <fake-ip> <ip>
Common vulnerable ports	nmap -p 445,3389,21,23 <ip>/<mask>
Detect ARP spoofing	arpwatch -i eth0
Perform ARP spoofing (audit)	sudo arpspoof -i eth0 -t <target-ip> <gateway-ip>
Sniff credentials (auth)	sudo tcpdump -i eth0 port 80 or port 21 -A

To use arpwatch and arpspoof, install:
`sudo apt install arpwatch dsniiff`

WI-FI AUDITING HANDSHAKE CAPTURE

Enable monitor mode	sudo airmon-ng start wlan0
Disable monitor mode	sudo airmon-ng stop wlan0mon
Scan nearby networks	sudo airodump-ng wlan0mon
Target specific BSSID/Channel	sudo airodump-ng --bssid <BSSID> -c <channel> wlan0mon
Capture handshake to file	sudo airodump-ng -c <channel> --bssid <BSSID> -w capture wlan0mon
Verify handshake in pcap	tshark -r capture-01.cap -Y "eapol"
Deauth test (audit client)	sudo aireplay-ng --deauth 5 -a <BSSID> -c <CLIENT_MAC> wlan0mon

To use airmon-ng, airodump-ng, aireplay-ng, and tshark, install:
`sudo apt install aircrack-ng tshark`